

Chapter 2

Data Models and DATABASE SYSTEM
ARCHITECTURE

Objective

- to briefly describe the evolution of database systems and to understand some of the main data models used and now in use.
- To understand the database system Architecture

Outline

- Basic Database Terminologies
- Overview of data models
- Schemas and Instances
- DBMS Architecture
- Data Independence

Basic Database Terminologies

- **Enterprise** - an organization : A library, a bank, a university, etc.
- **Entity** - person, place, thing, or event
 - An "object" in the real world that we are interested in like Student, Customer, Book, ...
- **Attribute (Field)** - a characteristics of an Entity, that has a specific meaning. Eg. Name, age, telephone, grade, sex, etc.
- **Record** - a logically connected set of one or more Attributes that describe an Entity
- **File** - a collection of related records. For example, a file might contain data about customers; or students of a certain department in a university
- **Database** - collection of Files

Data Model

- A set of primitives for defining the structure of a database.
- Database Model is a collection of tools or concepts for describing:
 - Data
 - Data relationships
 - Data semantics
 - Data constraint
- The main purpose of database model is to represent the data in an understandable way.

Data Model

- Categories of database models include:
 - Object-based
 - Record-based
 - Physical

Evolution of Major Data Models

GENERATION	TIME	MODEL	EXAMPLES	COMMENTS
First	1960s–1970s	File system	VMS/VSAM	Used mainly on IBM mainframe systems Managed records, not relationships
Second	1970s	Hierarchical and network	IMS ADABAS IDS-II	Early database systems Navigational access
Third	Mid-1970s to present	Relational	DB2 Oracle MS SQL-Server	Conceptual simplicity Entity relationship (ER) modeling support for relational data modeling
Fourth	Mid-1980s to present	Object-oriented Extended Relational	Versant VFS/FastObjects Objectivity/DB DB/2 UDB Oracle 10g	Support complex data Extended relational products support objects and data warehousing Web databases become common
Next Generation	Present to future	XML	dbXML Tamino DB2 UDB Oracle 10g MS SQL Server	Organization and management of unstructured data Relational and object models add support for XML documents

File-Processing Systems

- The earliest database systems – during the 1960s
 - Data was stored in *flat text files* in the operating system.
 - Data was also grouped into **records**
 - e.g. a record of Customer data in a bank might look like this:
Number, Name, FathersName, PhoneNumber, Address,
SavingsAccountNumber, SavingsAccountBalance,
CurrentAccountNumber, CurrentAccountBalance
- 123,Tesfay, Kebede, 04 407600, Yeka kefle ketema Addis Ababa,
1234, 1000, 9876, 1000
- A file contains data about **one entity** e.g. Customer is an entity. Each record in the file represents an **instance** of that entity.
 - Each **row** is a **record**; the file represents a **record type**.

Cont'd

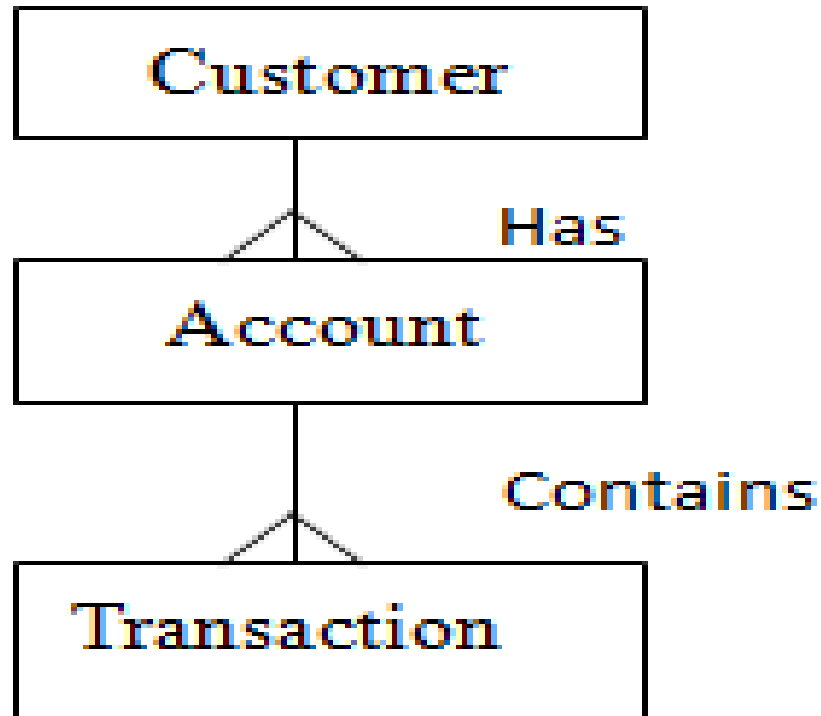
Disadvantages

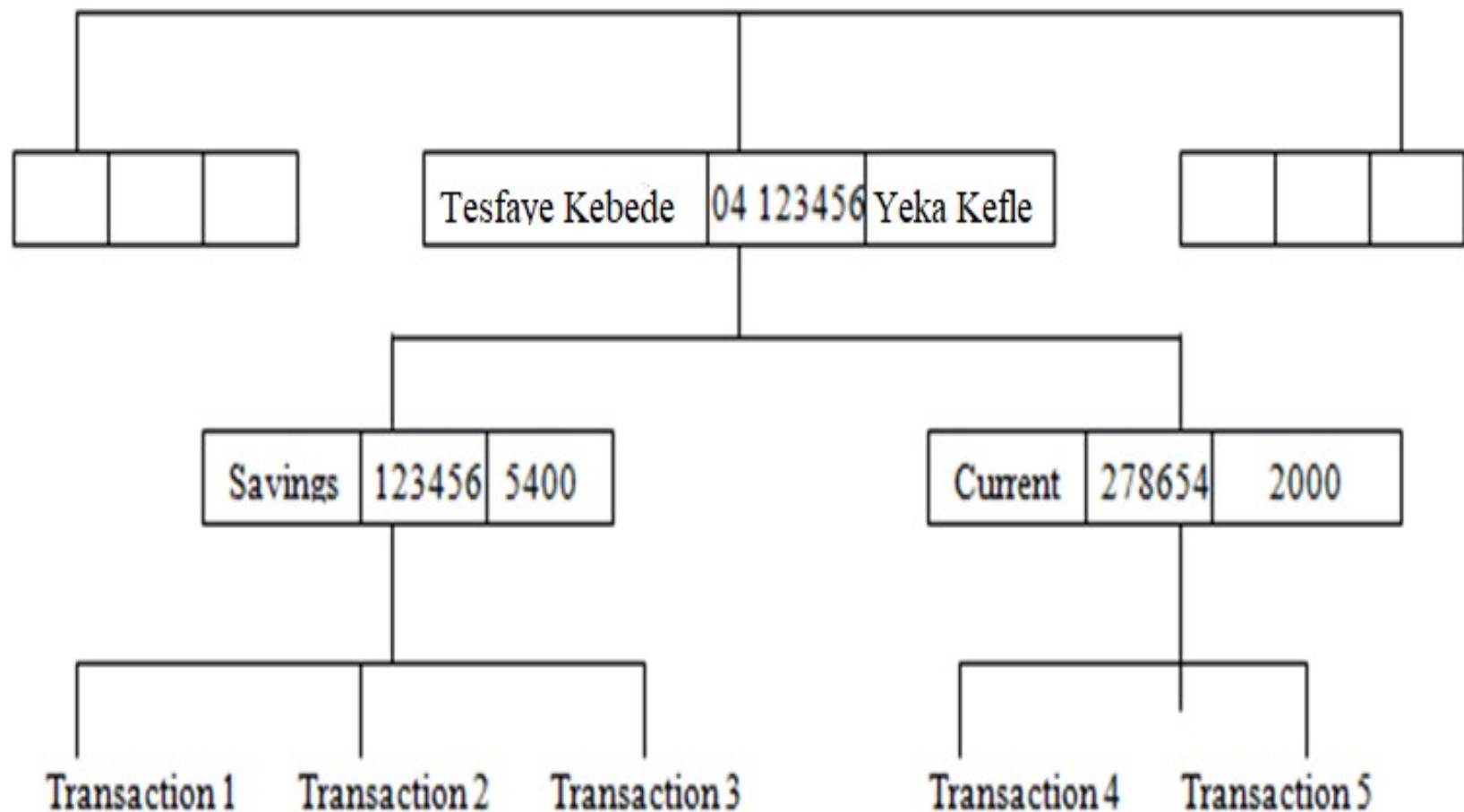
- Programmers had to write specific programs to carry out each task - so a lot of work was involved.
 - e.g. to retrieve all customer records or to find the customer record for a given customer number.
- It could also lead to data being **duplicated** in different files
 - e.g. think of a bank's database. A customer's name and phone number could appear in a file containing all savings accounts. If the same customer also has a current account, their name and phone number would also appear in the file containing all current accounts. Or if a customer has 2 savings accounts – need 2 rows in the file – name and phone number are repeated.
- If the customer changes their phone number – it has to be changed in both files.
- This system worked, but **didn't allow data to be related** – e.g. to have the customer name and phone number in one place, and relate a savings account to it and a current account to it.

Record-based Data Models

- Consist of a number of fixed format records.
- Each record type defines a fixed number of fields
- Each field is typically of a fixed length.
- There are three types of Record-based data models:-
 - Hierarchical Database Model
 - Network Database Model
 - Relational Database Model

Hierarchical Model





cont'd

- The simplest database model
- Records are represented as nodes or segments
 - Nodes are arranged in a hierarchical structure as sort of upside-down tree. The tree may be of arbitrary depth
- Each record contain multiple fields, where each field may contain either data values like integer, real, text or a pointer to a record. The pointer is not allowed to form a cycle. Some hierarchical DBMS support null values or variable-length fields .

Cont'd

- The top node is the **root node**
- The relationship between parent and child is *one-to-many and one-to-one*
 - A parent node can have more than one child node
 - A child node can only have one parent node
- Relation is established by creating physical link between stored records (each is stored with a predefined access path to other records)

Cont'd

- The data is still stored in a flat file....like previously...but now the flat file has 3 different record types in it and the record types are related:

Customer: 123, Teffaye , Abebe, 04 407600, Yeka Kefle ketema,

Account: Savings, 123456, 5400

Transaction:.....

Transaction:....

Account: Current,278654, 2000

Customer:

Advantages

- Hierarchical Model is simple to construct and operate on
- Conceptual simplicity – easy to understand the model layout
- **Data independence** (a change in a data type will be automatically cascaded throughout the database by the DBMS, thereby eliminating the need to make changes in the program segments that reference the changes data type)
- **Database integrity** – always a link between parent and child
- **Efficiency** – very efficient when it contains a large volume of data in 1:M relationships and whose relationships are fixed over time

Cont'd

- Searching of any record in a hierarchical tree is very fast since the hierarchical databases uses contiguous storage for hierarchical structures.
- Language is simple; uses constructs like GET, GET UNIQUE, GET NEXT, GET NEXTWITHIN PARENT etc.

Disadvantage

- **Complex implementation** – detailed knowledge of the physical data storage characteristics is required by the designers and programmers
- **Difficult to manage** – relocation of segments requires application changes
- Lacks structural independence
- Complex applications programming and use – programmers and end users must know precisely how the data are physically distributed within the database
- **Lack of standards** – no standard DDL and no DML

Cont'd

- Navigational and procedural nature of processing.
- A programmer still has to write the program to access the account information, but now the program navigates through the hierarchy e.g. to find the balance of the savings account for a given customer number.
- does not support much consistency and security

Cont'd

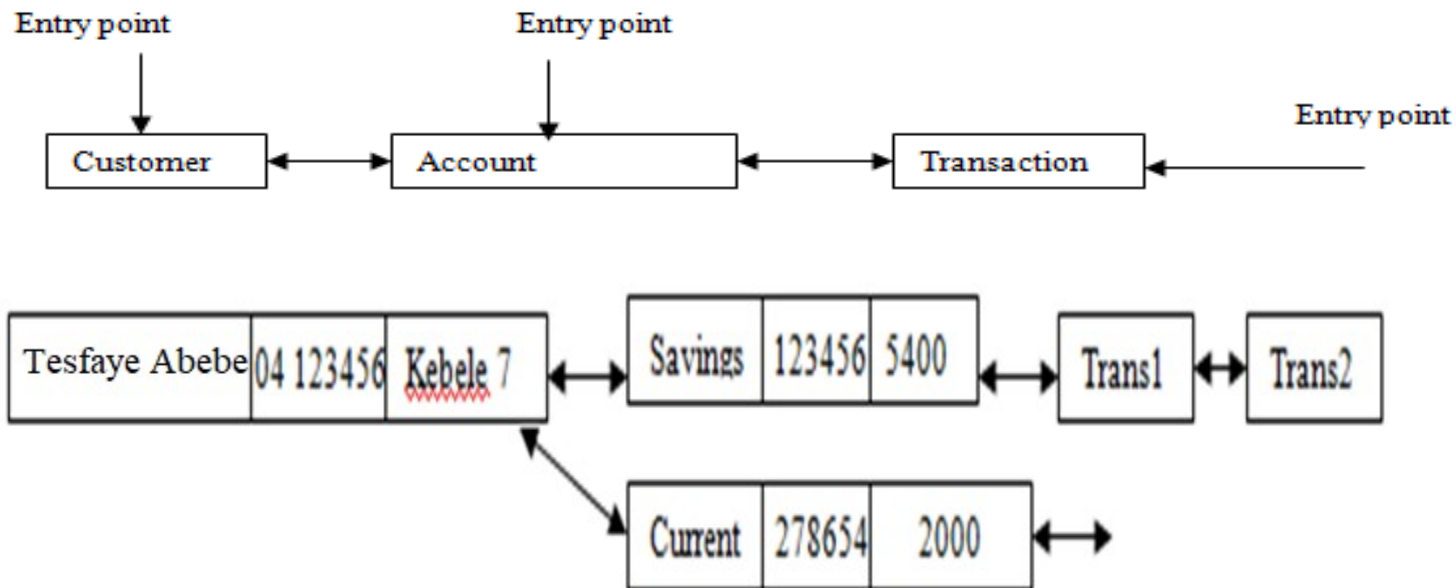
- Implementation limitations – difficult to support M:N relationships
 - a child node in the tree cannot have **more than one parent**. So if the same account is associated with 2 customers (e.g. a husband and wife have a joint account and also separate accounts), we cannot link the one account record to 2 different customer records. But we can have the account record appearing in two tree branches. This can lead to **duplicated data** – and inconsistent data, if the account is not updated in all the branches.

Cont'd

- To add new record type or relationship, the database must be redefined and then stored in a new form.

Network Model

- also uses records and links but in a different way.
- Instead of putting data in a hierarchy, it structures data in a network



Customer, Account and Transaction → *record types*.

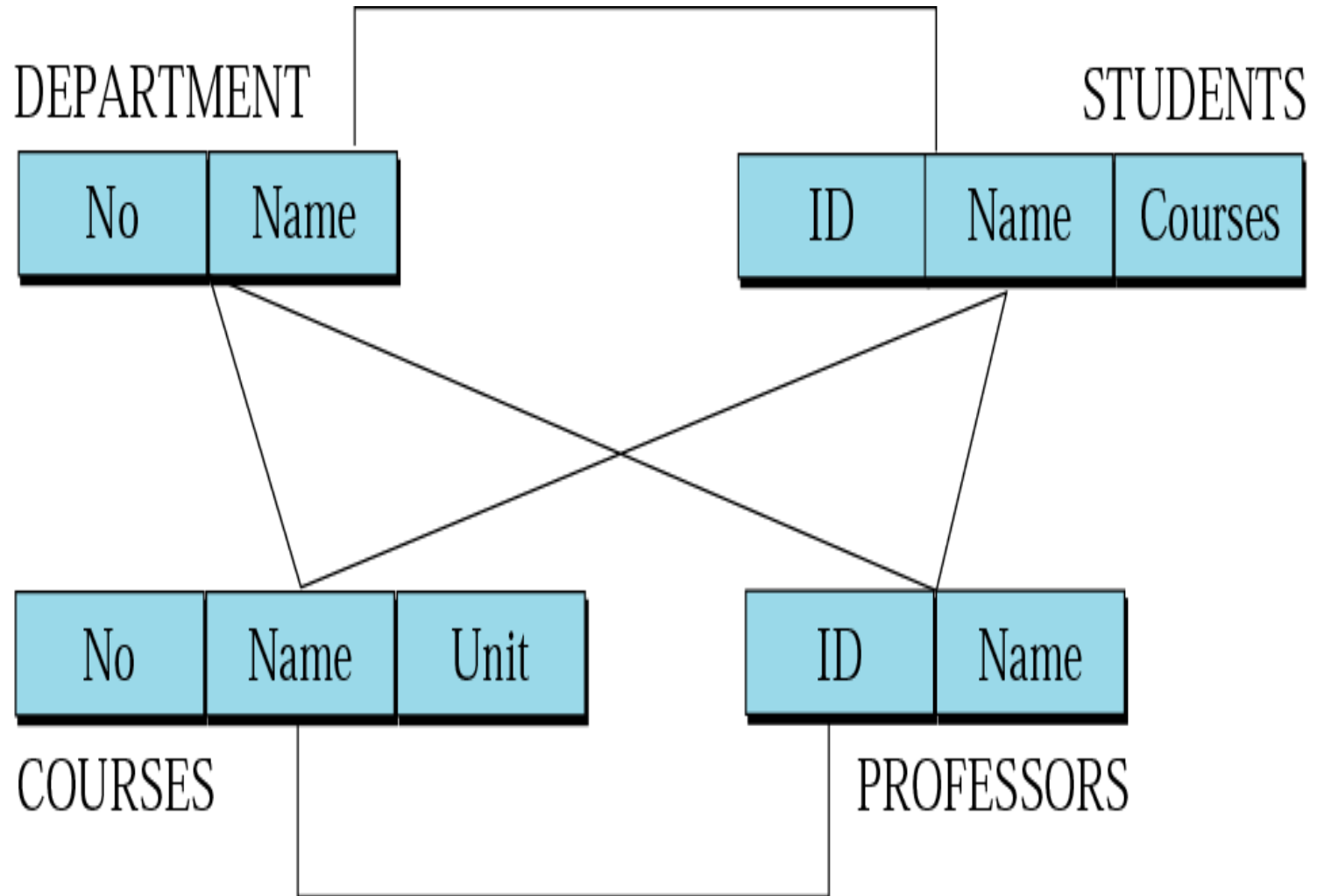
Cont'd

- Like hierarchical model network model is a collection of physically linked records.
- Collection of **records** in 1:M relationships
 - Relationships are modelled using **sets**.
 - In a set, there is one **owner**-record type--Equivalent to the hierarchical model's parent eg. (Customer) and 1 or more member record types Equivalent to the hierarchical model's child eg.(Account).This is a **set type**, let us name it CustomerAccount. It models the relationship between Customer & Account – that a Customer can have 1 or more accounts. There will be many occurrences of the CustomerAccount set in the database – one for each Customer. A set occurrence relates one record from the owner record type (Customer) to the set of records from the member record type related to it (Account).For each CustomerAccount set, there is a network of links from the Customer record to the Account records.
 - A record can appear as a member in more than one set i.e., a member may have multiple owners
- Can you identify another set type in this model?

Cont'd

- The data is still stored in **files** but now we also have to define the set types.
- The ***entry points*** are records that can be searched. An entry point is implemented as an **index** on a list of records – so the index must be created before such queries can be run against the database.
- In terms of data structure can you think of how entry implemented ?

Registrar system



Advantages

- Conceptual simplicity
- Data access flexibility – no need for a preorder traversal
- Promotes database integrity – must first define the owner and then the member record
- Data independence
- Network Model is able to model complex relationships and represents semantics of add or delete on the relationships.
- Can handle most situations for modeling using record types and relationship types.
- Conformance to standards

Disadvantages

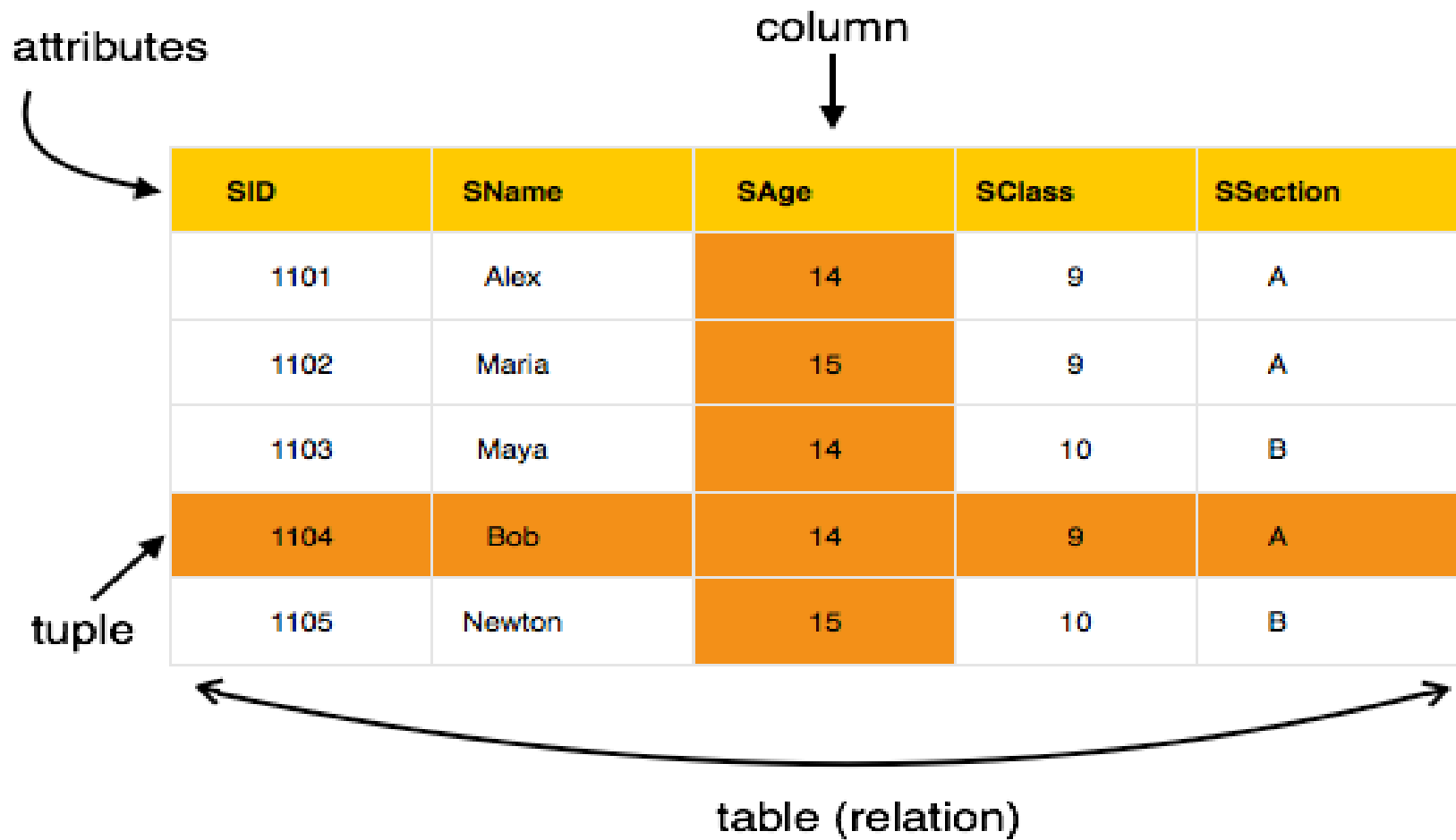
- System complexity
- Lack of structural independence
- Navigational and procedural nature of processing.
 - Language is navigational; uses constructs like FIND, FIND member, FIND owner, FIND NEXT within set, GET etc.
Programmers can do optimal navigation through the database
- Database contains a complex array of pointers that thread through a set of records.
- Little scope for automated "query optimization"

Relational Model

- Developed by Dr. Edgar Frank Codd in 1970 (famous paper, 'A Relational Model for Large Shared Data Banks').
- Terminologies originates from the branch of mathematics called **set theory** and **relational algebra**.
- Can define more flexible and complex relationship.
- Viewed as a collection of tables called “Relations” equivalent to collection of record types.

Relation

- Two dimensional table.
- Stores information or data in the form of **tables rows and columns**.
 - A **row** of the table is called **tuple** equivalent to **record**.
 - A column of a **table** is called **attribute** equivalent to **fields**.



Alternative terminologies

Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

Cont'd

- **Data value** is the value of the Attribute.
- **Records** are related by the data **stored jointly in the fields of records** in two tables or files. The related tables contain information that creates the relation.
- No physical consideration of the storage is required by the user.
- Many tables are merged together to come up with a new **virtual view** of the relationship.
- The relational data model is implemented through a very sophisticated **relational database management system (RDBMS)**.

Cont'd

Customers

<u>CustomerID</u>	Name	<u>PhoneNumber</u>	Address
1	Tesfaye Kebede	04 123456	Yeka Kefle Ketema
2			

Accounts

Type	Number	Balance	<u>CustomerID</u>
Savings	123456	5400	1
Current	278654	2000	1

Cont'd

- The RDBMS manages all of the physical details, while the user sees the relational database as a collection of tables in which data are stored.
- The user can manipulate and query the data in a way that seems intuitive and logical.

- Conducts searches by using data in specified columns of one table to find additional data in another table.
- In conducting searches, a relational database model matches information from a field in one table with information in a corresponding field of another table to produce a third table that combines requested data from both tables.

Advantages

- **Structural independence** – changes in the relational data structure do not affect the DBMS's data access in any way.
- Improved conceptual simplicity by concentrating on the **logical view** .
- **Easier** database design, implementation, management, and use.
- Ad hoc query capability – SQL.
- Use of powerful database management system.

Disadvantages

- Substantial hardware and system software overhead.
- Can facilitate poor design and implementation.

Object relational

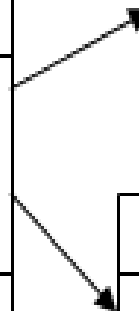
- An Object relational model is a combination of a Object oriented database model and a Relational database model. So, it supports **objects, classes, inheritance** etc. just like Object Oriented models and has support for **data types, tabular structures** etc. like Relational data model.
- One of the major goals of Object relational data model is to close the gap between relational databases and the object oriented practices frequently used in many programming languages such as C++, C#, Java etc.

Customer
<u>CustomerID</u> Name <u>PhoneNumber</u> Address
<u>newCustomer</u> <u>removeCustomer</u> ...

Account
<u>AccountNumber</u> Balance <u>CustomerID</u>
<u>lodgeMoney</u> <u>withdrawMoney</u>

SavingsAccount
<u>InterestRate</u>

CurrentAccount
<u>OverdraftLimit</u>
<u>cashCheque</u>



Advantages

- **Inheritance**
 - The Object Relational data model allows its users to inherit objects, tables etc. so that they can extend their functionality. Inherited objects contains new attributes as well as the attributes that were inherited.
- **Complex Data Types**
 - Complex data types can be formed using existing data types. This is useful in Object relational data model as complex data types allow better manipulation of the data.
- **Extensibility**
 - The functionality of the system can be extended in Object relational data model. This can be achieved using complex data types as well as advanced concepts of object oriented model such as inheritance.

Disadvantages

The object relational data model can get quite **complicated** and difficult to handle at times as it is a combination of the Object oriented data model and Relational data model and utilizes the functionalities of both of them.

Object Oriented Approach

- Object oriented data model is based upon real world situations. These situations are represented as objects, with different attributes. All these object have multiple relationships between them.
- **Elements of Object oriented data model**
 - **Objects**
 - The real world entities and situations are represented as objects in the Object oriented database model.
 - **Attributes and Method**
 - Every object has certain characteristics. These are represented using Attributes. The behavior of the objects is represented using Methods.

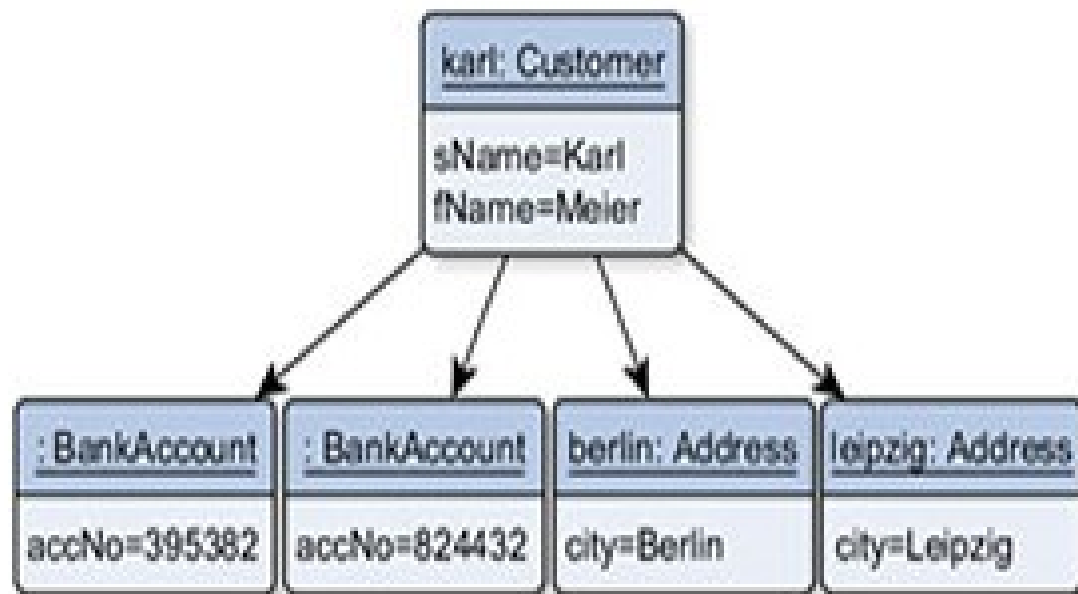
– **Class**

- Similar attributes and methods are grouped together using a class. An object can be called as an instance of the class.

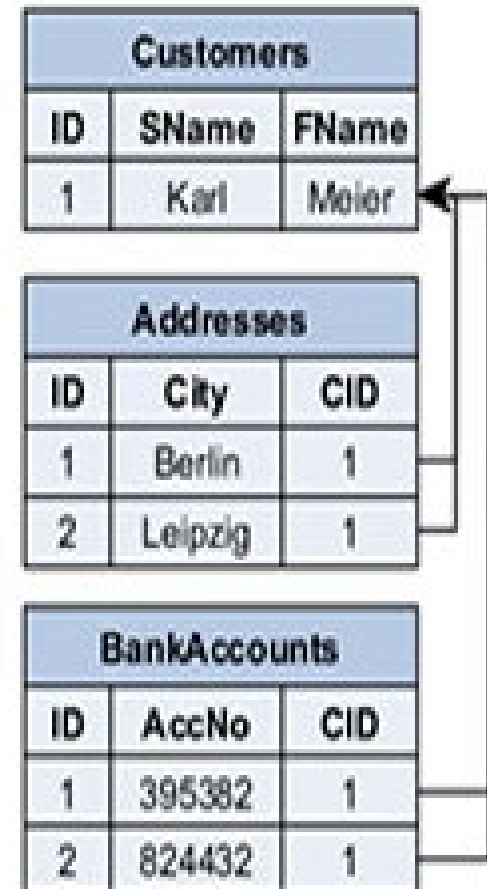
– **Inheritance**

- A new class can be derived from the original class. The derived class contains attributes and methods of the original class as well as its own.

Object-Oriented Data Model



Relational Data Model



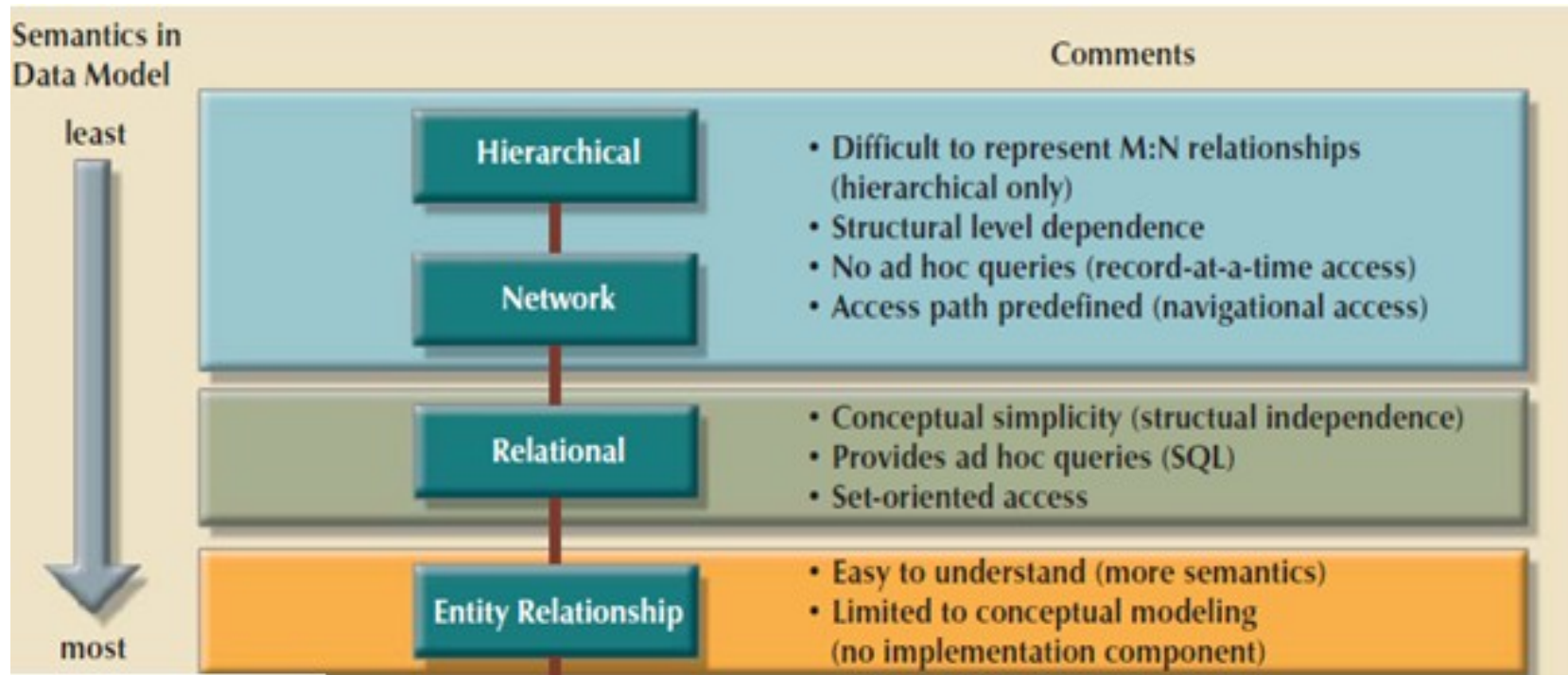
Deductive Model

- In a deductive db system, rules can be defined – the rules *deduce* or infer additional information from the facts stored in the database.
- Deductive databases are really a type of **knowledge base**, used in the area of AI (Artificial Intelligence).
- A deductive db has ***facts*** and ***rules*** in it. Facts are stored similar to relations in a relational database, but **attribute names** are not necessary.
- Rules are specifications that can be applied to the facts – to produce new information.

Cont'd

- The rules are defined using a declarative language (what, rather than how...).
- The system has an *inference engine* that deduces new facts from the db by interpreting the rules.
- The deductive model is closely related to the relational model; it also has its basis in a branch of **mathematics** (domain relational calculus).
- In a deductive database system, the emphasis is on deriving new knowledge from existing data by supplying rules based on knowledge of the real world.

Summary of Data Model



Schema & Instance

There are two basic components of the database: the definition of the Relation (Schema) and the actual data stored in each table (Instance)

Database Schema

- Describes how data is to be structured, defined at setup/design time (also called "metadata")
- it specifies name of relation, the name of each field and the type of each field
- Example:
 - ***Students(sid: string, name: string, login: string, age: integer, gpa: real)***
- The preceding schema says that each record in the Students relation has five fields, with field names and types as indicated

Database Schema

- The structure of a database that
 - Captures data types, relationships and constraints in data
- Is independent of any application program
- Changes infrequently

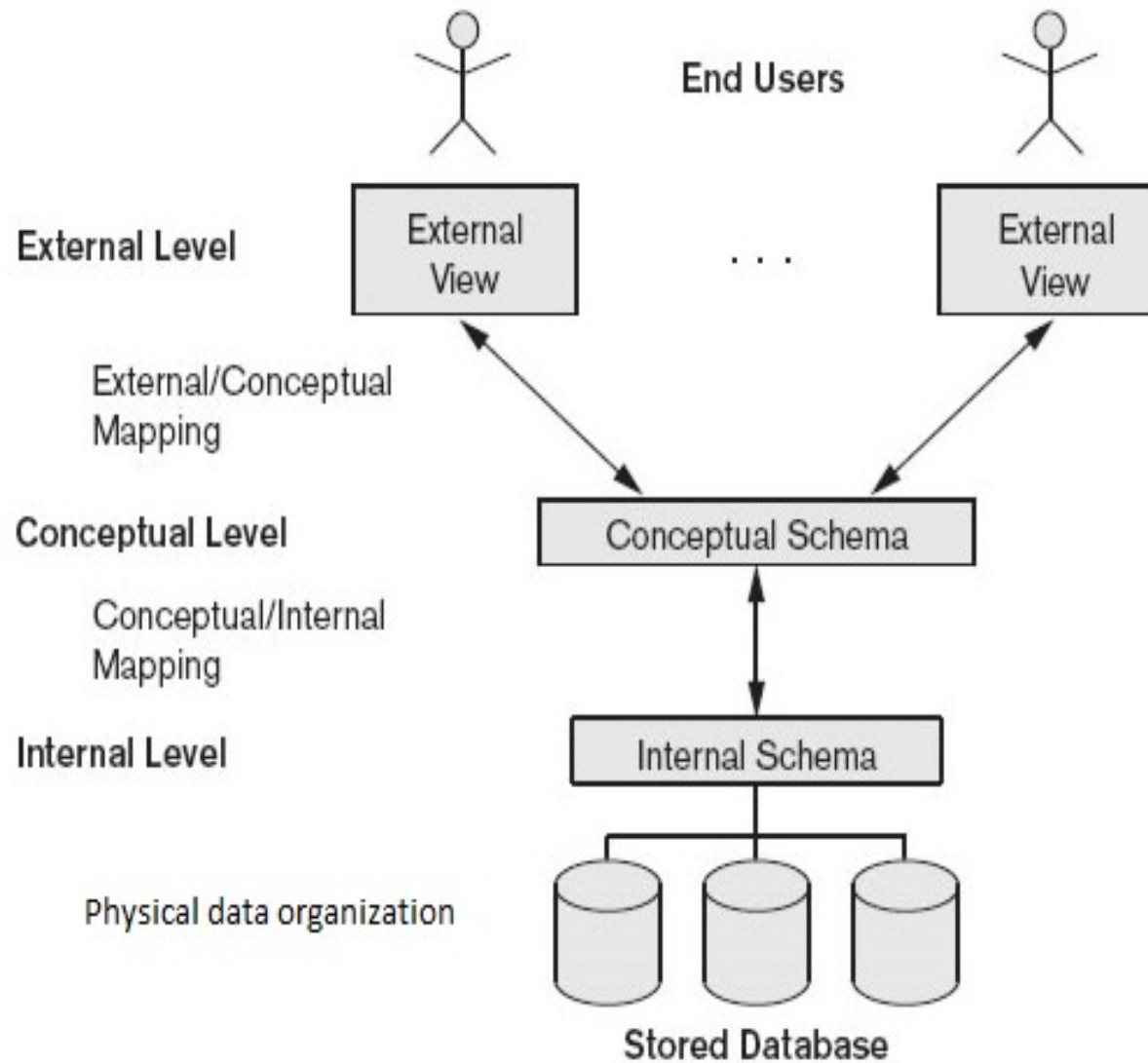
Database Instance or State:

- is the collection of the actual data contained in the database at a particular point of time
- Also called **State** or **Snap Shot** or **Extension** of the database
- Since Instance is actual data of database at some point in time, it changes rapidly
- To define a new database, we specify its database schema to the DBMS (database is empty), database is initialized when we first load it with data

Three levels ANSI_SPARC Architecture

- The Systems Planning and Requirements Committee of the American National Standards Institute encapsulated the concept of schema in its **three-level** database architecture model, known as the ANSI/SPARC architecture, which is shown in the diagram below

Three-level ANSI-SPARC Architecture of a Database System



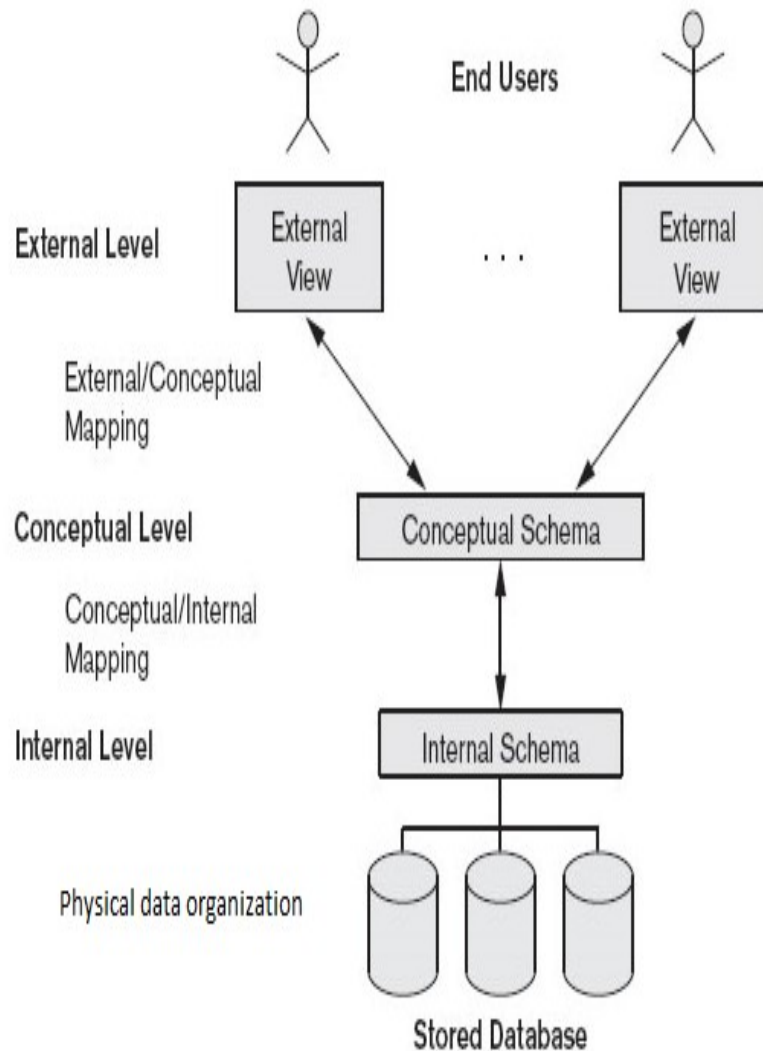
ANSI/SPARC three-level architecture

- ANSI = American National Standards Institute
- ANSI/X3 = Committee on Computers and Information Processing
- SPARC = Standards Planning and Requirements Committee
- The ANSI/SPARC model is a three-level database architecture with a hierarchy of levels, from the users and their applications at the top, down to the physical storage of data at the bottom.

- The Architecture of most of commercial DBMS are available today is mostly based on this ANSI-SPARC database architecture
- Proposed to support DBMS characteristics of:
 - **Program-data independence**
 - Support of **multiple views** of the data

Objective of Three-level ANSI_SPARC architecture

- It allows independent customized user views:
 - Each user should be able to access the same data, but have a different customized view of the data
 - These should be independent: changes to one view should not affect others
- It hides the physical storage details from users:
 - Users should not have to deal with physical database storage details
- Database Administrator should be able to change conceptual structure of database without affecting all users
- Database Administrator should be able to change database storage structures without affecting the users' views



- **External Level**
 - Users' view of the database
- **Conceptual Level**
 - Community view of the database
- **Internal Level**
 - Physical representation of the database on the computer

Cont'd

External Level

- The **external** or **view level** includes a number of external schemas or user views.
- Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.
- It excludes irrelevant data as well as data which the user is not authorized to access.
- Implemented using a representational data mode

Cont'd

Conceptual Level

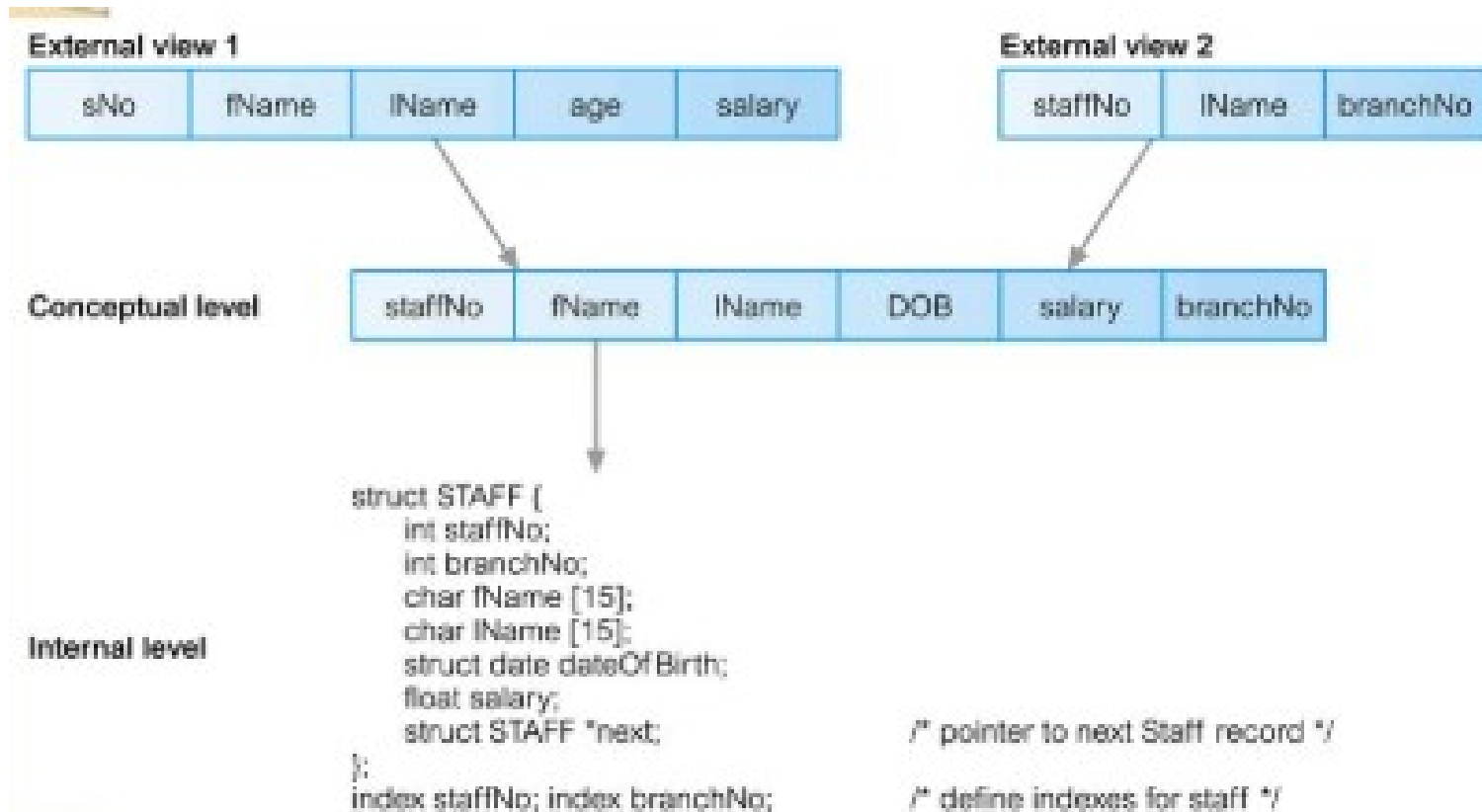
- The conceptual level is a way of describing what data is stored within the whole database and how the data is inter-related
- The **conceptual level** has a **conceptual schema**, which describes the structure of the whole database for a community of users.
- concentrates on describing entities, data types, relationships, user operations, and constraints.
- Usually, a representational data model is used to describe the conceptual schema when a database system is implemented.
- This level does not specify how the data is physically stored

Cont'd

Internal Level

- The internal level has an internal schema, which describes the physical storage structure of the database.
- The internal level involves how the database is physically represented on the computer system
- It describes how the data is actually stored in the database and on the computer hardware
- The internal schema uses a physical data model

Three levels ANSI_SPARC Architecture Schema Example



External level

sno	Iname	fname	age	sal
-----	-------	-------	-----	-----

sno	fname	Br.No.
-----	-------	--------

Conceptual level



Sno	fname	Iname	DOB	Sal.	Br. No.
------------	--------------	--------------	------------	-------------	----------------

Internal level

```
Struct STAFF{  
  Int staff_no;  
  Char fname[15];  
  Char lname[15];  
  Struct date date_of_birth;  
  Float sal;  
  Struct staff *next  
};
```

Three levels ANSI_SPARC Architecture Schema Example

- External Schema (View):
 - *Course_info(cid,sid,name)*
- Conceptual (logical) schema:
 - *Students(sid: string, name: string, login: string, age: integer, gpa:real)*
 - *Courses(cid: string, cname:string, credits:integer)*
 - *Enrolled(sid:string, cid:string, grade:string)*
- Internal(Physical) schema:
 - Relations stored as unordered files
 - Index on first column of Students

Database schemas

- There are three different types of schema corresponding to the three levels in the ANSI-SPARC architecture
- The external schemas
 - describe the different external views of the data and there may be many external schemas for a given database
- The conceptual schema
 - describes all the data items and relationships between them of the whole database
 - There is only one conceptual schema per database
- The Internal schema
 - describes physical storage structures and access paths
 - There is only one internal schema per database
- The overall description of a database is called the *database schema*

Data Independence

- A very important advantage of using a DBMS is that it offers **Data Independence**
- That is, application programs are insulated from changes in the way the data is structured and stored
- Data independence is achieved through use of the three levels of data abstraction (i.e., External level, Conceptual level & Internal level)

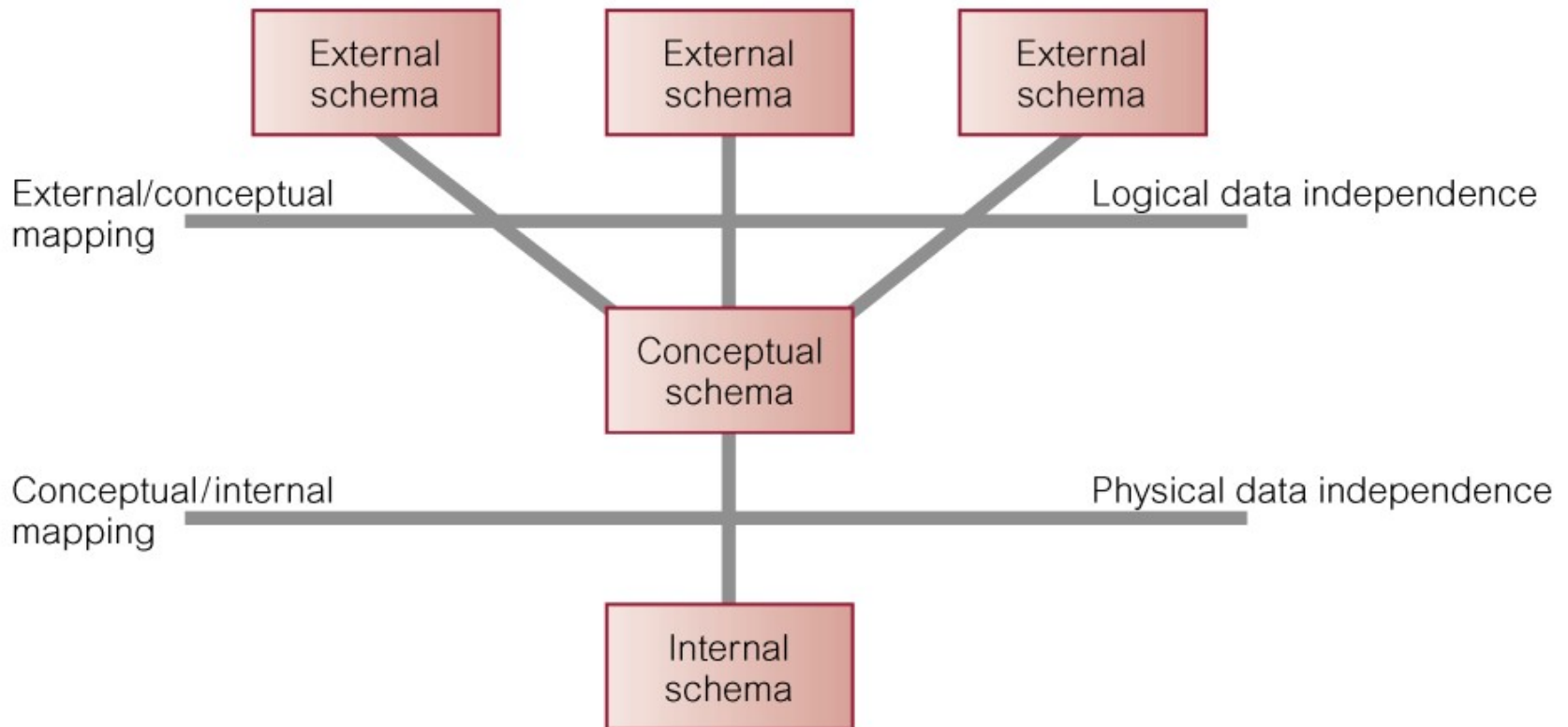
Logical Data Independence

- The capacity to change the conceptual schema without having to change the external schemas and their application programs
- Conceptual schema changes e.g. addition/removal of entities should not require changes to external schema or rewrites of application programs
- Here are some examples of changes in the logical layer that can be safely made thanks to logical data independence:
 - Adding a new database object
 - Adding data items to an existing object

Physical Data Independence

- The ability to modify the internal schema without changing the conceptual schema
- The conceptual schema hides details such as how the data is actually laid out on disk, the file structure, and the choice of indexes
- Physical schema changes e.g. using different file organizations, storage structures/devices should not require change to conceptual or external schemas

Data Independence and the ANSI/SPARC Three-Level Architecture



Two-Tier Client-Server Architecture

- Client manages main business and data processing logic and user interface
- Server manages and controls access to database
- Disadvantages:
 - 'Fat' client, requiring considerable resources on client's computer to run effectively
 - Significant client side administration overhead



Three-Tier Client-Server Architecture

- a type of multi-tier computing architecture in which an entire application is distributed across three different computing layers or tiers
 - User interface layer – runs on client
 - Business logic and data processing layer – middle tier runs on a server (application server)
 - DBMS – stores data required by the middle tier. This tier may be on a separate server (database server)
- Advantages:
 - ‘Thin’ client, requiring less expensive hardware
 - Application maintenance centralized
 - Easier to modify or replace one tier without affecting others

